CHyLL v2: Continuous Latent Embeddings of Hybrid Systems

Method, loss terms, and a guide to the trained models and data

Kaito Iwasaki

Hybrid Dynamical Systems

A *hybrid* dynamical system mixes smooth motion with sudden jumps.

- A passive walker: the swing leg moves smoothly, then strikes the ground and the state jumps.
- A bouncing ball: free flight under gravity, then an instantaneous change of velocity at each impact.

The jump is a genuine discontinuity. A smooth model of the dynamics cannot be fitted directly across it.

Three Example Systems

System	State dim.	Behavior
Rimless wheel	2	stable rolling limit cycle
Bouncing ball	2	bounces, losing energy at each impact
Compass-gait biped	4	period-doubling route to chaos

They increase in difficulty. The compass-gait biped walks with period 1, then 2, 4, 8, and finally chaotically as the ground slope steepens.

The Idea

Replace the discontinuous system with a continuous one that is easy to model.

- An **encoder** maps each state into a latent space.
- A single **smooth vector field** advances the latent state in time.
- A **decoder** maps the latent state back.

The jump is absorbed into the smooth latent flow. This follows Teng, Liu, and Sreenath (ICML 2026).

The Mapping-Cylinder State Space

To make the jump continuous, the state is augmented with one extra coordinate $s \in [0, 1]$:

$$y = (x, s) \in X'.$$

- $s = 0$: ordinary motion of the original system.
- s sweeping $0 \to 1$: the jump, unrolled as a continuous path.

On X' the trajectory is continuous, so a smooth latent model can be fitted to it.

The Learned Maps

CHyLL v2 learns three maps. Write $Z = \mathbb{R}^d$ for the latent space.

$$
\begin{aligned}
E_\theta &: X' \to Z && \text{encoder,} \\
V_\theta &: Z \to Z && \text{latent vector field,} \\
D_\theta &: Z \to X' && \text{decoder.}
\end{aligned}
$$

The time-τ flow of V_θ is written F_θ. The latent dimension is $d = 7$ for the rimless wheel and bouncing ball, and $d = 11$ for the compass-gait biped.

Training: The Forward Pass

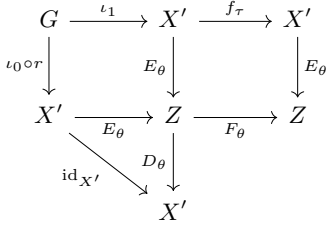
For a sampled trajectory slice $y_0, \ldots, y_{T-1}$:

$$
\begin{aligned}
z_i &= E_\theta(y_i), \\
\widehat{z}_i &= F_\theta^i(z_0), \\
\widehat{y}_i &= D_\theta(\widehat{z}_i).
\end{aligned}
$$

Each state is encoded, the latent flow is integrated forward, and the integrated latent trajectory is decoded. The loss compares the encoded z_i, the integrated $\widehat{z}_i$, and the decoded $\widehat{y}_i$ against the data.

The Commutative Diagram

Here f_τ is one time-τ step of the system, G the impact set, r the reset map, and ι_0, ι_1 the inclusions at $s = 0$ and $s = 1$. The learned maps should make this diagram commute:



Each cell is enforced by one loss term:

- Left square, L_g: E_θ sends both sides of a reset to one latent point.
- Top-right square, L_z: the latent flow matches the dynamics.
- Lower triangle, L_x: the decoder inverts the encoder.

Two Conditions Beyond the Diagram

Commuting is not the whole story. Two more terms act at the seam where s wraps from 1 back to 0:

- L_v: the latent velocity has no kink crossing a seam.
- L_c: the encoder keeps all d latent coordinates active and does not collapse them.

These seams are detected from the sampled coordinate s in the data, not from symbolic guard equations.

What Each Loss Term Enforces

Term	Enforces	Code helper
L_x	decoder recovers the state	`reconstruction_loss`
L_z	encode/flow square commutes	`latent_consistency_loss`
L_g	reset pairs share a latent point	`gluing_loss`
L_v	latent velocity continuous at seams	`seam_velocity_loss`
L_c	latent coordinates do not collapse	`anti_collapse_loss`

L_g, L_z, and L_x are the three commuting cells of the diagram. L_v and L_c are the two conditions beyond it.

The Composite Objective

$$L = w_x L_x + w_z L_z + w_g L_g + w_v L_v + w_c L_c.$$

Setting	$(w_x, w_z, w_g, w_v, w_c)$
Rimless wheel, bouncing ball	$(1,\ 1,\ 3,\ 1,\ 1)$
Compass-gait biped	$(1,\ 1,\ 3,\ 0,\ 1)$

The exact values for any run are stored in its `config.json`.

Results

- **Rimless wheel.** The model reproduces the rolling limit cycle, and the learned embedding recovers its expected single topological cycle.
- **Compass-gait biped.** The cascade analysis recovers the period-doubling sequence exactly: the learned latent states form 2, 4, and 8 clusters at the period-2, -4, and -8 gaits, and disperse under chaos.
- **Bouncing ball.** The orbit spirals slowly toward rest rather than closing into a loop, so its topological signature is only partially recovered.

Where the Models and Data Live

Path	Contents
`runs/<name>/`	trained model, config, training log
`figures/<name>/`	loss and rollout figures
`cycling_signature/data/`	exported lifts and rank summaries
`cycling_signature/figures/`	rank distribution and heatmap figures
`compass_analysis/`	compass cascade cluster analysis

Paths are under `chyll_v2/`. Each `<name>` is one trained run, for example `rimless_wheel_phaseB_finetune`.

A Trained Run

Each directory in `chyll_v2/runs/` contains:

- `model.pt`: the learned network weights.
- `config.json`: dimensions, system parameters, and loss weights.
- `train_log.jsonl`: the loss terms recorded over training.

The config is enough to rebuild the network and reload the weights. The export scripts under `cycling_signature/export/` do exactly this.

What an Exported Lift Contains

An exported lift is a sampled trajectory in the latent space Z:

- `..._positions`: the sampled latent points, one per row.
- `..._tangents`: the unit tangent direction at each point.

Both arrays have shape (N, d): N samples and d latent coordinates. Each lift is written as `.npy` for Python and `.csv` for inspection.

Loading a Lift in Python

```
import numpy as np

folder = "chyll_v2/cycling_signature/data/rimless_wheel"
base = folder + "/continuous_lift_chyll_v2_phaseB"

Z = np.load(base + ".npy")            # positions
T = np.load(base + "_tangents.npy")  # unit tangents

print(Z.shape, T.shape)
print(np.linalg.norm(T, axis=1).max())
```

`Z` holds the latent positions and `T` the unit tangents. The printed tangent norm should be close to 1.

From a Trained Model to a Rank Figure

The topology analysis runs in three stages:

1. **Export**: a script in `cycling_signature/export/` turns a trained model into a latent lift.
2. **Compute**: `cycling_signature/julia/run_subsegments.jl` computes the cycling signature over random subsegments.
3. **Plot**: `cycling_signature/plot/plot_signature_figures.py` draws the rank distribution and heatmaps.

The `cycling_signature/` folder has its own README with the exact commands.

Sanity Checks

After loading a lift:

- positions and tangents have the same shape,
- tangent norms are close to 1,
- the latent dimension d matches the system, 7 or 11.

A common mistake is pairing a model with the wrong `config.json`: the network shape will not match the saved weights. Take `model.pt` and `config.json` from the same run directory.

Where the Code Lives

Object	File
$E_\theta, V_\theta, D_\theta$	`networks.py`
latent ODE rollout	`ode.py`
loss terms L_x, L_z, L_g, L_v, L_c	`losses.py`
training loop	`train.py`
system simulators and reset maps	`systems/`

These files are the CHyLL v2 core library, under `chyll_v2/chyll_v2/`. Topology postprocessing lives under `chyll_v2/cycling_signature/`.

Summary

- A hybrid system is made continuous on the mapping cylinder X'.
- CHyLL v2 learns an encoder, a latent vector field, and a decoder.
- L_g, L_z, and L_x are the commuting cells of the diagram.
- L_v and L_c add the seam-regularity conditions.
- The learned embeddings recover the expected topology for the rimless wheel and the compass-gait cascade.